

OBJECT ORIENTED PROGRAMMING

CS305PC: OBJECT ORIENTED PROGRAMMING
THROUGH JAVA



NEED FOR OOP PARADIGM

- OOP is an approach to program organization and development, which attempts to eliminate some of the drawbacks of conventional programming methods by incorporating the best of structured programming features with several new concepts.
- OOP allows us to decompose a problem into number of entities called objects and then build data and methods (functions) around these entities.
- The data of an object can be accessed only by the methods associated with the object.

OOP PARADIGMS

- Some of the Object-Oriented Paradigm are:
- Emphasis is on data rather than procedure.
- Programs are divided into objects.
- Data Structures are designed such that they
 - Characterize the objects.
- Methods that operate on the data of an object are tied together in the data structure.
- Data is hidden and can not be accessed by external functions.
- Objects may communicate with each other through methods.

SUMMARY OF OOPS

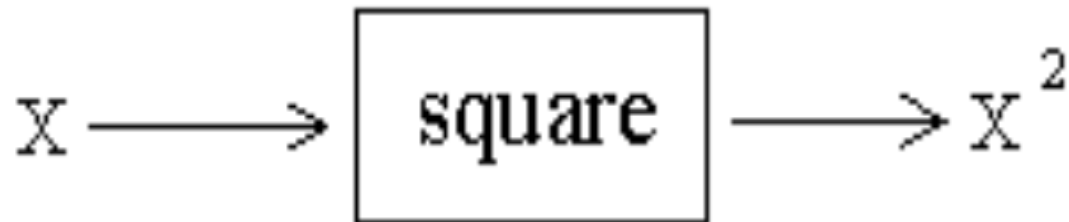
- The following are the basic oops concepts: They are as follows:
- Objects.
- Classes.
- Data Abstraction.
- Data Encapsulation.
- Inheritance.
- Polymorphism.
- Dynamic Binding.
- Message Passing.

COPING WITH COMPLEXITY

- **"Coping with complexity"** in OOP is all about breaking down large problems into smaller, manageable objects using *abstraction, encapsulation, inheritance, and polymorphism*. It makes code easier to read, reuse, debug, and maintain.
- **Design Patterns & Principles:**
 - Using standard solutions like **MVC**, **Singleton**, or **Factory Pattern** helps structure complex code effectively.
 - Following **SOLID** principles (Single Responsibility, Open/Closed, etc.) makes systems easier to understand and evolve.

ABSTRACTION IN OOP (PROCEDURAL ABSTRACTION)

- Procedural Abstractions organize instructions.
- Let's first address the problem of squaring a number. Instead of having to write the expression $r * r$ every time, it would be nice to have a black box, or a procedure, that would take in a numerical argument and return the square of that argument.
- In Java, we can create a procedure called square as follows:
- **`double square(double x) { return (x * x); }`**
- If we had this, we could write
- **`square(837)`**



ABSTRACTION IN OOP (DATA ABSTRACTION)

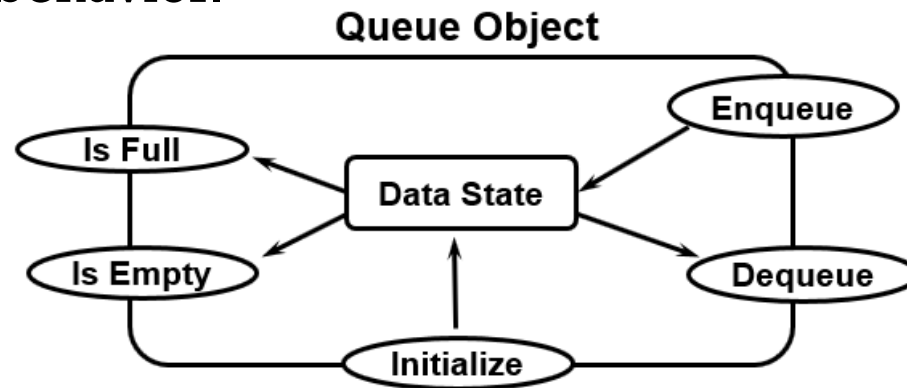
- Data Abstractions organize data.
- **Data types.** A data type is a set of values and a set of operations on those values.
- **Abstract data types (ADT).** An abstract data type is a data type whose internal representation is hidden from the client
- **Example:** Java's *String* data type is an important and useful ADT.

StudentType

Name (string)
Marks (num)
Grade (char)
Student Number (num)

ABSTRACTION IN OOP (BEHAVIORAL ABSTRACTION)

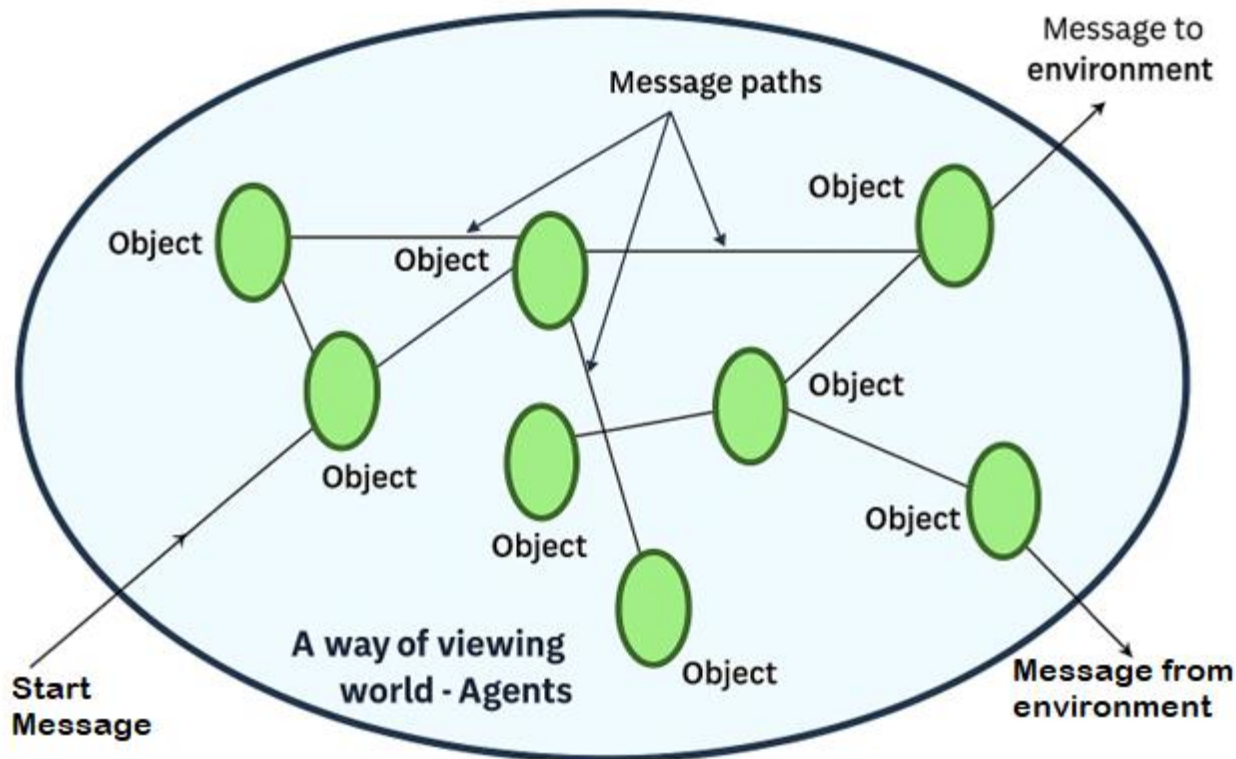
- Behavioral Abstractions combine procedural and data abstractions.
- Behavioral abstraction, also known as **control abstraction**, focuses on hiding the complex implementation details of how an operation or method is performed. It defines a contract for behavior without specifying the exact steps involved in achieving that behavior.



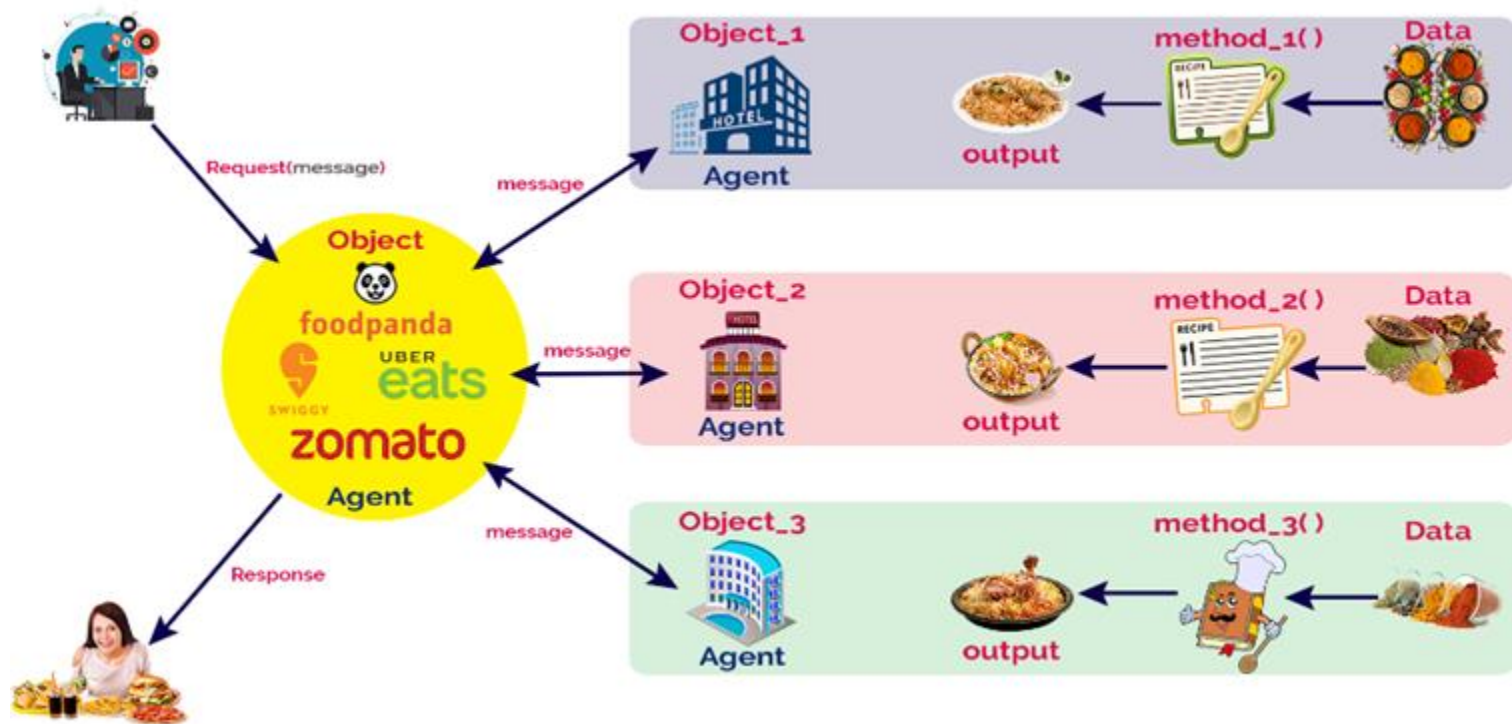
A WAY OF VIEWING WORLD — AGENTS

- OOP uses an approach of treating a real-world agent as an object.
- Object-oriented programming organizes a program around its data (that is, objects) and a set of well-defined interfaces to that data.
- An object-oriented program can be characterized as data controlling access to code by switching the controlling entity to data.

MESSAGE PASSING AMONG OBJECTS IN A PROGRAM SHELL



A WAY OF VIEWING WORLD WITH OOP



RESPONSIBILITY

- primary motivation is the need for a platform-independent (that is, architecture- neutral) language that could be used to create software to be embedded in various consumer electronic devices, such as microwave ovens and remote controls.
- Objects with clear responsibilities
- Each class should have a clear responsibility.
- If you can't state the purpose of a class in a single, clear sentence, then perhaps your class structure needs some thought.

MESSAGES

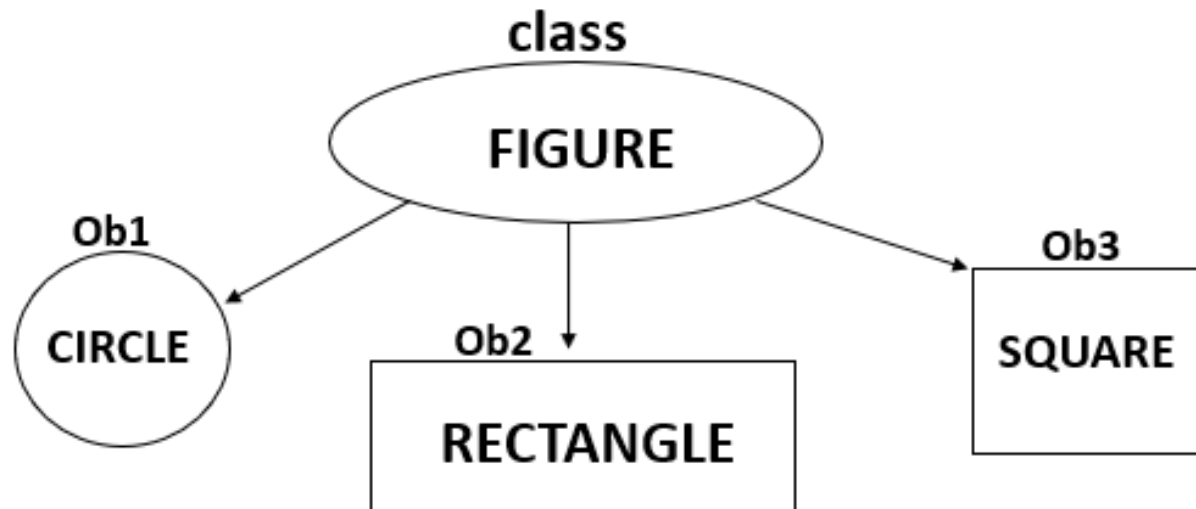
- We all like to use programs that let us know what's going on. Programs that keep us informed often do so by displaying status and error messages.
- These messages need to be translated so they can be understood by end users around the world.
- The Section discusses translatable text messages. Usually, you're done after you move a message String into a ResourceBundle.
- If you've embedded variable data in a message, you'll have to take some extra steps to prepare it for translation.

METHODS

- A method is a group of instructions that is given a name and can be called up at any point in a program simply by quoting that name.
- Drawing a Triangle require draw of three straight lines. This instruction three times to draw a simple triangle.
- We can define a method to call this instruction three times and draw the triangle(i.e. create a method drawLine() to draw lines and this method is called repeatedly to achieve the needed task)
- The idea of methods appears in all programming languages, although sometimes it goes under the name functions and sometimes under the name procedures.
- The name methods is a throw-back to the language C++, from which Java was developed.
- In C++, there is an object called a class which can contain methods. However, everything in Java is enclosed within a class .so the functions within it are called methods

CLASSES

- Class is blue print or an idea of an Object
- From One class any number of Instances can be created
- It is an encapsulation of attributes and methods



SYNTAX OF CLASS

```
class <ClassName>{  
    attributes/variables;  
    Constructors();  
    methods();  
}
```


INSTANCE

- Instance is an Object of a class which is an entity with its own attribute values and methods.
- Creating an Instance

```
ClassName refVariable;
```

```
refVariable = new Constructor();
```

or

```
ClassName refVariable = new Constructor();
```

JAVA CLASS HIERARCHY

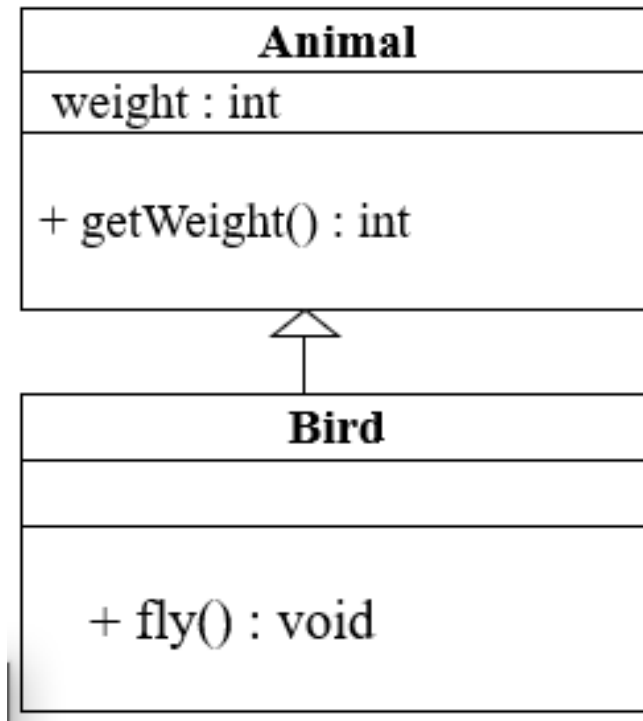
- In Java, class “Object” is the base class to all other classes
 - If we do not explicitly say extends in a new class definition, it implicitly extends Object
 - The tree of classes that extend from Object and all of its subclasses are is called the class hierarchy
 - All classes eventually lead back up to Object
 - This will enable consistent access of objects of different classes.

INHERITANCE

- Methods allows to reuse a sequence of statements
- Inheritance allows to reuse classes by deriving a new class from an existing one
- The existing class is called the parent class, or superclass, or base class
- The derived class is called the child class or subclass.
- The child class inherits characteristics of the parent class(i.e the child class inherits the methods and data defined for the parent class

INHERITANCE

- Inheritance relationships are often shown graphically in a class diagram, with the arrow pointing to the parent class



METHOD BINDING

- Objects are used to call methods.
- Method Binding is an object that can be used to call an arbitrary public method, on an instance that is acquired by evaluating the leading portion of a method binding expression via a value binding.
- It is legal for a class to have two or more methods with the same name.
- Java has to be able to uniquely associate the invocation of a method with its definition, relying on the number and types of arguments.
- Therefore the same-named methods must be distinguished:
 - 1) by the number of arguments, or
 - 2) by the types of arguments
- Overloading and inheritance are two ways to implement polymorphism.

METHOD OVERRIDING.

- There may be some occasions when we want an object to respond to the same method but have different behaviour when that method is called.
- That means, we should override the method defined in the superclass. This is possible by defining a method in a sub class that has the same name, same arguments and same return type as a method in the superclass.
- Then when that method is called, the method defined in the sub class is invoked and executed instead of the one in the superclass. This is known as overriding.

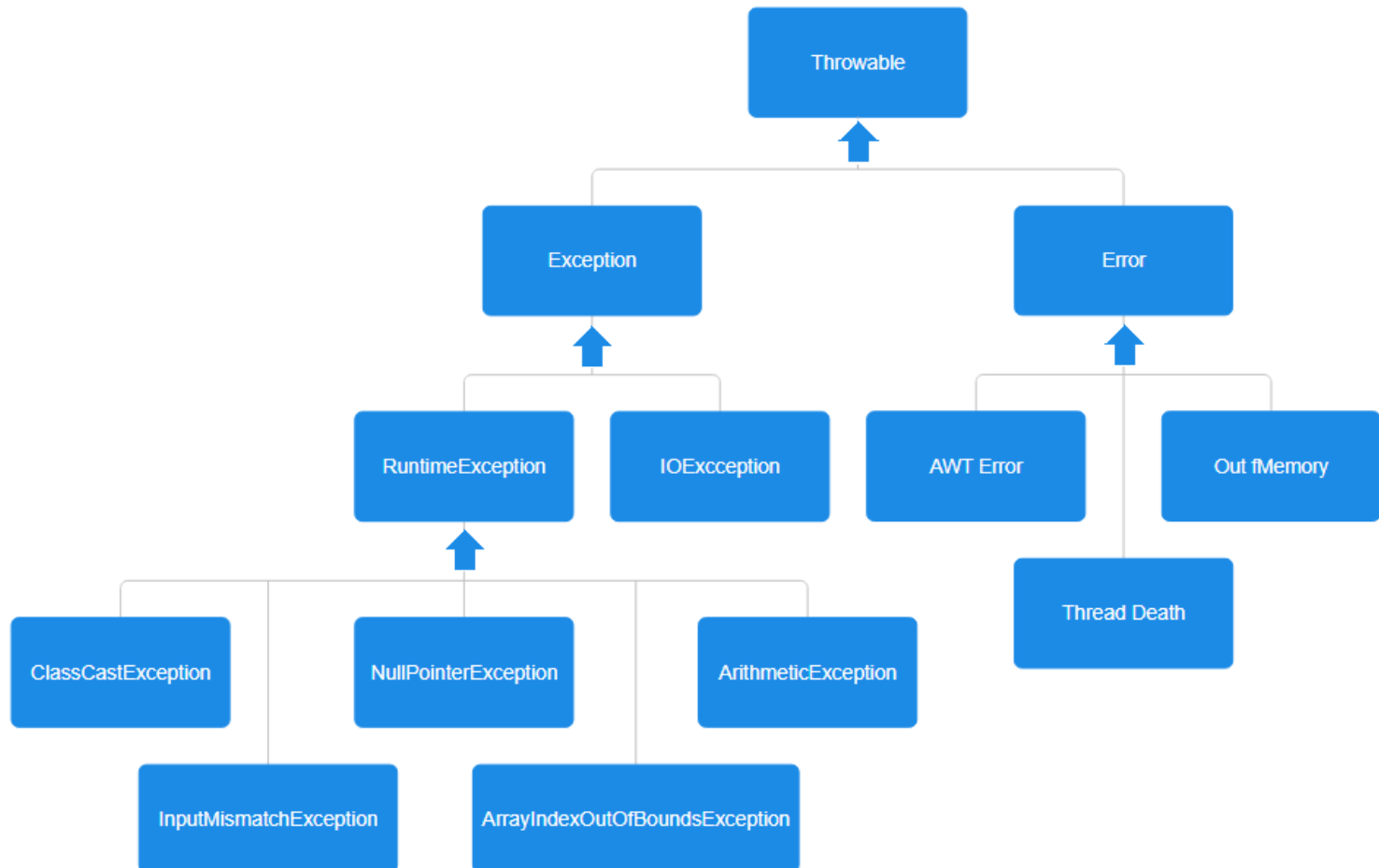
EXCEPTIONS IN JAVA

- Exception is an abnormal condition that arises in the code sequence.
- Exceptions occur during compile time or run time.
- “throwable” is the super class in exception hierarchy.
- Compile time errors occurs due to incorrect syntax.
- Run-time errors happen when
 - User enters incorrect input
 - Resource is not available (ex. file)
 - Logic error (bug) that was not fixed

EXCEPTION CLASSES

- In Java, exceptions are objects. When you throw an exception, you throw an object. You can't throw just any object as an exception, however -- only those objects whose classes descend from Throwable.
- Throwable serves as the base class for an entire family of classes, declared in java.lang, that your program can instantiate and throw.
- Throwable has two direct subclasses, Exception and Error.
- Exceptions are thrown to signal abnormal conditions that can often be handled by some catcher, though it's possible they may not be caught and therefore could result in a dead thread.
- Errors are usually thrown for more serious problems, such as OutOfMemoryError, that may not be so easy to handle. In general, code you write should throw only exceptions, not errors.
- Errors are usually thrown by the methods of the Java API, or by the Java virtual machine itself.

EXCEPTION CLASS HIERARCHY



HISTORY OF JAVA

- Created By: James Gosling and team at Sun Microsystems
- First Released: 1995
- Original Name: Oak (renamed to Java in 1995)
- Purpose: Initially developed for embedded systems and digital devices (like set-top boxes)
- Key Milestones:
 - 1991 Project started as "Oak"
 - 1995 Renamed to Java, first public release (Java 1.0)
 - 1998 Java 2 (J2SE) introduced
 - 2006 Java became open source (OpenJDK)
 - 2010 Oracle acquired Sun Microsystems
 - 2017 Switch to time-based release cycle (every 6 months)

JAVA BUZZWORDS (PRINCIPLES)

- Java was designed with certain principles (buzzwords) that made it powerful and platform-independent.

Buzzword	Meaning
Simple	Easy to learn; similar to C/C++ but with less complexity
Object-Oriented	Everything is treated as an object
Robust	Strong memory management, exception handling, and type checking
Secure	Built-in security features like bytecode verification and sandboxing
Platform-Independent	"Write once, run anywhere" via Java Virtual Machine (JVM)
Portable	Java programs can run on different architectures without modification
High Performance	Just-In-Time (JIT) compiler helps speed up performance
Multithreaded	Supports multiple threads for concurrent programming
Distributed	Built-in support for networking and remote method invocation
Dynamic	Supports dynamic linking and runtime class loading

JAVA KEYWORDS

- Java has a fixed set of reserved words (keywords) that cannot be used as identifiers (variable/class names).
- List of All 67 Java Keywords (as of Java 17):

<code>abstract</code>	<code>continue</code>	<code>for</code>	<code>new</code>	<code>switch</code>
<code>assert</code>	<code>default</code>	<code>goto*</code>	<code>package</code>	<code>synchronized</code>
<code>boolean</code>	<code>do</code>	<code>if</code>	<code>private</code>	<code>this</code>
<code>break</code>	<code>double</code>	<code>implements</code>	<code>protected</code>	<code>throw</code>
<code>byte</code>	<code>else</code>	<code>import</code>	<code>public</code>	<code>throws</code>
<code>case</code>	<code>enum</code>	<code>instanceof</code>	<code>return</code>	<code>transient</code>
<code>catch</code>	<code>extends</code>	<code>int</code>	<code>short</code>	<code>try</code>
<code>char</code>	<code>final</code>	<code>interface</code>	<code>static</code>	<code>void</code>
<code>class</code>	<code>finally</code>	<code>long</code>	<code>strictfp</code>	<code>volatile</code>
<code>const*</code>	<code>float</code>	<code>native</code>	<code>super</code>	<code>while</code>

'goto' and 'const' are reserved but not used in Java.